

SLE-2: PROFILING REPORT

BFS vs DFS Performance Analysis

Course	02AML204 – Introduction to Artificial Intelligence
PRN	25UAM137
Name	Harshal kamble
Division	B
Date	22/09/2026

1. Algorithms / Versions Profiled

- Algorithm A: Breadth-First Search (BFS)
- Algorithm B: Depth-First Search (DFS)
- Problem used: Complete binary-tree graph with 32,767 nodes and tree depth 14.
- Start node: 0
- Target node: 32766

2. Profiling Method

- Tool used: Python `time.perf_counter()` and a manual node counter.
- Measurement method: Both algorithms were executed on the same graph, start node, and target node. Execution time was recorded in milliseconds and the number of expanded nodes was counted.
- Number of runs: 5

3. Results

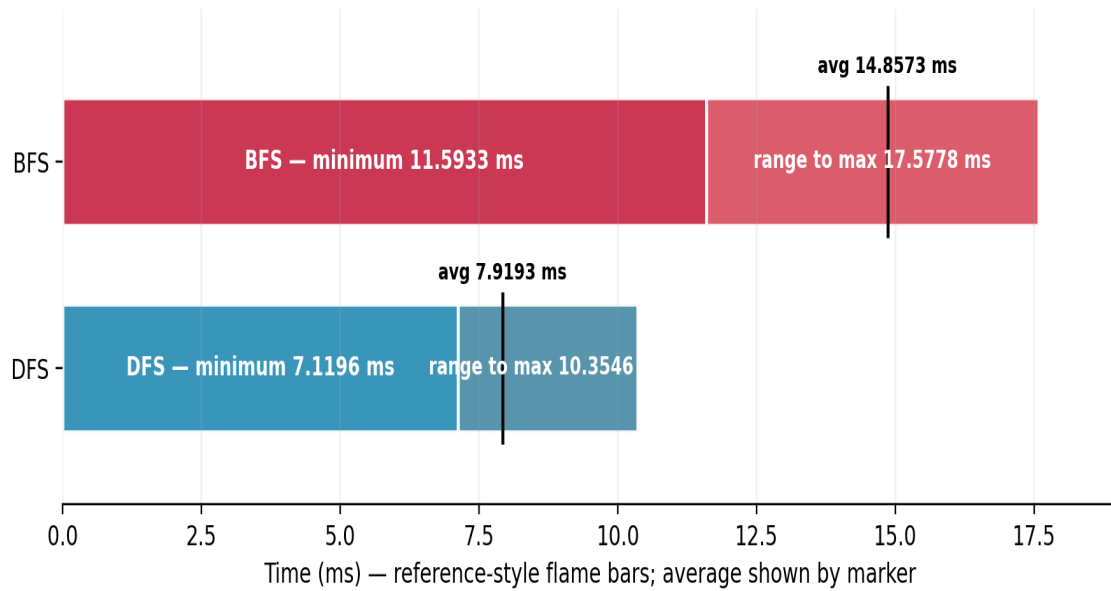
Metric	BFS	DFS	Observation
Average Time (ms)	14.8573	7.9193	DFS faster
Minimum Time (ms)	11.5933	7.1196	DFS lower
Maximum Time (ms)	17.5778	10.3546	DFS lower
Nodes Expanded	32,767	32,767	Equal

Short observation: BFS recorded an average time of 14.8573 ms, while DFS recorded 7.9193 ms. Both algorithms expanded 32,767 nodes in this experiment. DFS was faster according to the measured average execution time.

4. Charts

Figure 1: py-spy flame graph

Fig. 1 — Py-spy-style flame graph



Uses the report's minimum, maximum and average timings; no function-level stack data were supplied.

Figure 2: Nodes expanded

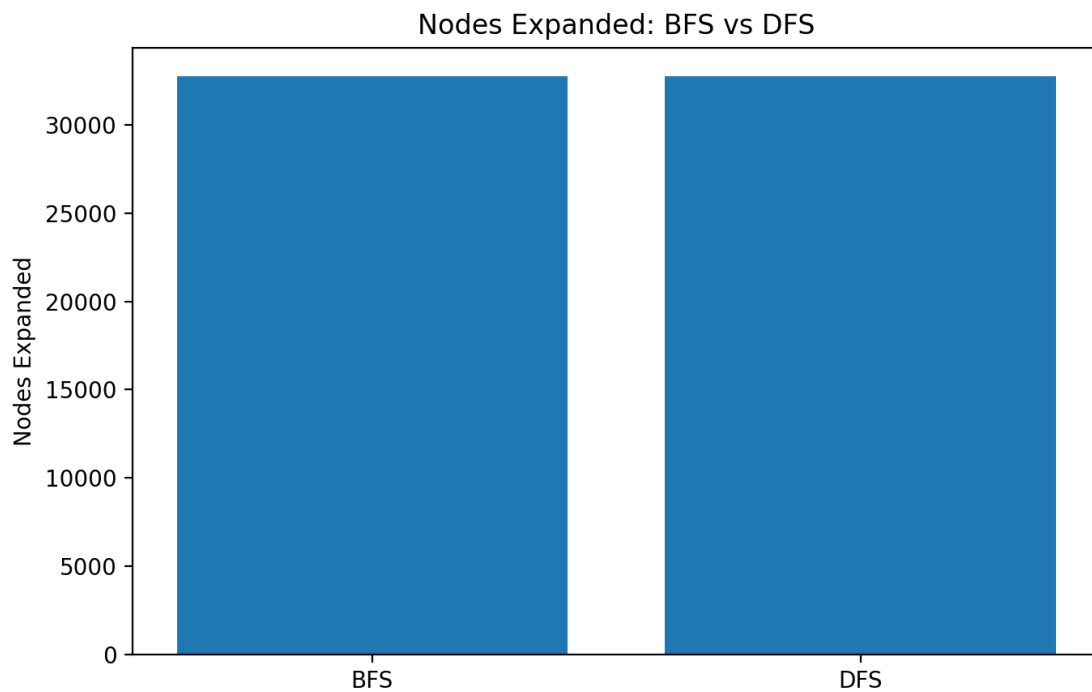
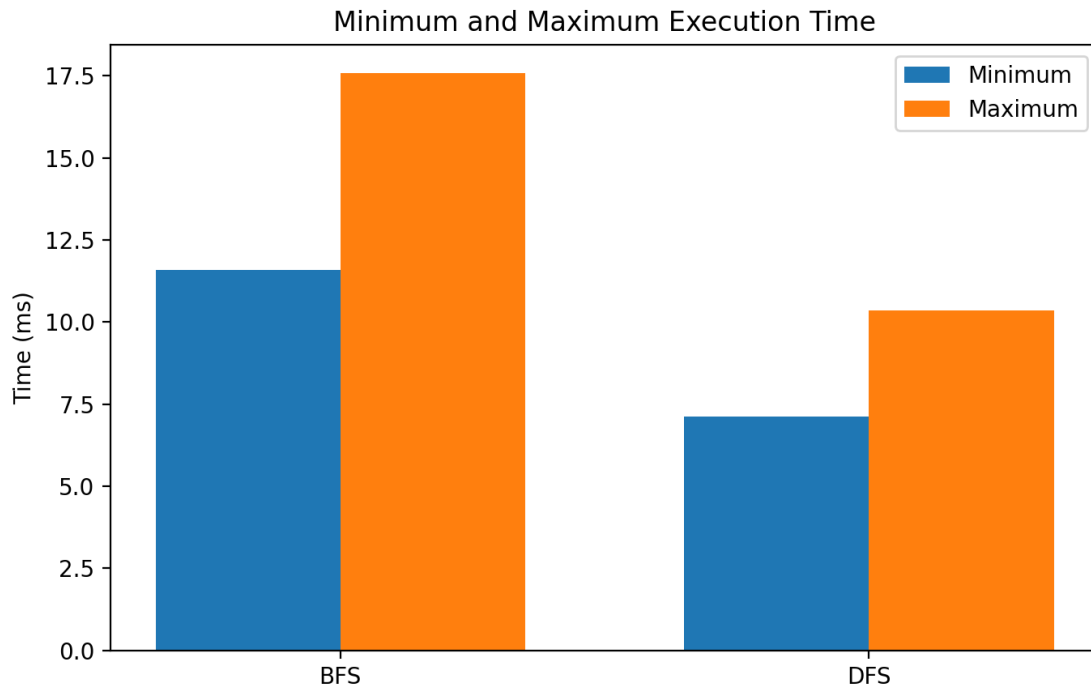


Figure 3: Minimum and maximum execution time



5. Justification & Analysis

BFS and DFS were tested on the same binary-tree graph with the same start and target nodes. BFS recorded an average execution time of 14.8573 ms, whereas DFS recorded 7.9193 ms. Both algorithms expanded 32,767 nodes before reaching the target. DFS was faster in this particular experiment according to the measured average time. The equal node count shows that the number of expanded nodes alone does not fully explain the execution-time difference. Implementation details, queue or stack operations, and processing overhead can affect the measured time. The result is specific to the selected graph structure and target node and should not be treated as a universal ranking of BFS and DFS.

6. AI Contribution Note

- AI tool used: ChatGPT.
- What AI helped with: Understanding the SLE-2 requirements, preparing the BFS and DFS implementation, explaining profiling concepts, and organizing the report.
- What I did myself: Ran the Python program, collected the terminal results, and used the measured values to prepare the comparison table and analysis.

7. Conclusion

This profiling exercise helped me understand the practical performance of BFS and DFS. Both algorithms expanded 32,767 nodes in the selected experiment. DFS recorded a lower average execution time (7.9193 ms) than BFS (14.8573 ms). I learned how to use execution-time measurement and a manual node counter to compare algorithms using real data. The results also show that performance depends on the graph structure, target location, and implementation.